

Лекция 6

Темы

0) Вопросы по теме предыдущих лекций, организационные

1) Django (Джанго) — свободный фреймворк для веб-приложений на языке Python, использующий шаблон проектирования MVC. Обзор фреймворка, установка, простейшие программы

2) Работа с сокетами

Перенимая лучшее...

<https://github.com/TonikX/ITMO ICT WebDevelopment 2020-2021>

☰ README.md

ITMO ICT WebDevelopment 2020-2021

Репозиторий для реализации дистанционного обучения по дисциплине "Web-программирование".

Лекция 1.1 - Концепции разработки веб сервисов.

Презентация с лекции [тут](#).

Дополнительные материалы:

1. Иерархия компьютерных информационных систем для разработки сайта (+)
2. Топ-5 наиболее популярных CMS: какую выбрать? (++)
3. Веб-фреймворки: введение для новичков (классификация фреймворков) (++)
4. Чем отличаются фронтенд- и бэкэнд-разработка (+)
5. Что такое MVC: базовые концепции и пример приложения (++)

Лекция 1.2 - Компоненты клиент-серверного взаимодействия.

Презентация лекции [тут](#).

Дополнительные материалы:

1. Сетевая модель OSI (+)
2. Адресация в сетях (+)
3. TCP vs UDP (+++)

Фреймворки

- заготовки, шаблоны для программной платформы, определяющие архитектуру программной системы;
- программное обеспечение, облегчающее разработку и объединение разных модулей программного проекта.

Преимущества фреймворков

- Фреймворки приятны и удобны в использовании.
- Фреймворки позволяют быстрее создавать MVP. Это особенно важно для небольших, малоопытных компаний и организаций. Быстрый старт может сыграть определяющую роль в успехе продукта.
- Фреймворки помогают обходить особенности и баги платформы.
- «Если я знаю [вставьте название фреймворка], значит, мне за это будут платить.» Ваше владение новинками и лучшими фреймворками может сыграть большую роль при поиске работы.

Классификация фреймворков

По языку программирования:

- HTML/CSS-фреймворки (Bootstrap, Semantic UI, Foundation).
- PHP-фреймворки (Yii, Symphony, Zend, Kohana, CodeIgniter).
- Python-фреймворки (Django, Plone, Twisted, Flask, Tornado).
- ruby-фреймворки (Ruby on Rails (явный лидер), Sinatra, Padrino).
- java-фреймворки (Spring MVC, JSF, GWT).

Бэкенд-фреймворки:

- выше перечисленные фреймворки, кроме HTML/CSS.

frontend-фреймворки:

- jQuery
- AngularJS
- React
- Meteor

Front end и back end

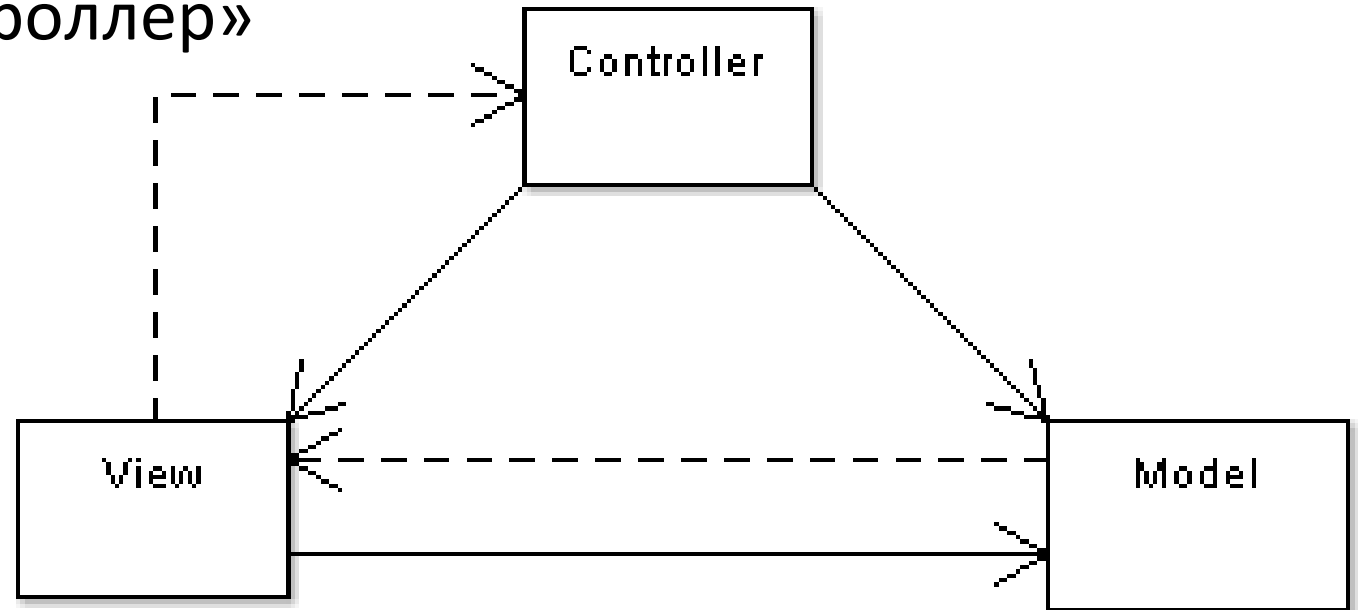
- **Фронтенд (англ. front-end)** — клиентская сторона пользовательского интерфейса к программно-аппаратной части сервиса.
- **Бэкенд (англ. back-end)** — программно-аппаратная часть сервиса.

Концепция MVC

- MVC (Model-View-Controller) - это конструкционный шаблон, который описывает способ построения структуры приложения, сферы ответственности и взаимодействие каждой из частей в данной структуре.
- **Преимущество, которое получают от использования концепции MVC - это чёткое разделение логики представления (интерфейса пользователя) и логики приложения.**

Концепция MVC

- Model-View-Controller
- «Модель-вид-контроллер»
- «Модель-представление-поведение»,
- «Модель-представление-контроллер»



Обработка данных и логика приложения

➤ **Model**

Представление данных пользователю в
любом поддерживаемом формате

➤ **View**

Обработка запросов пользователя и
вызов соответствующих ресурсов

➤ **Controller**

MVC

Модель (Model)

Собственно данные, методы для работы с данными, изменения и обновления данных.

Представление/Вид (View)

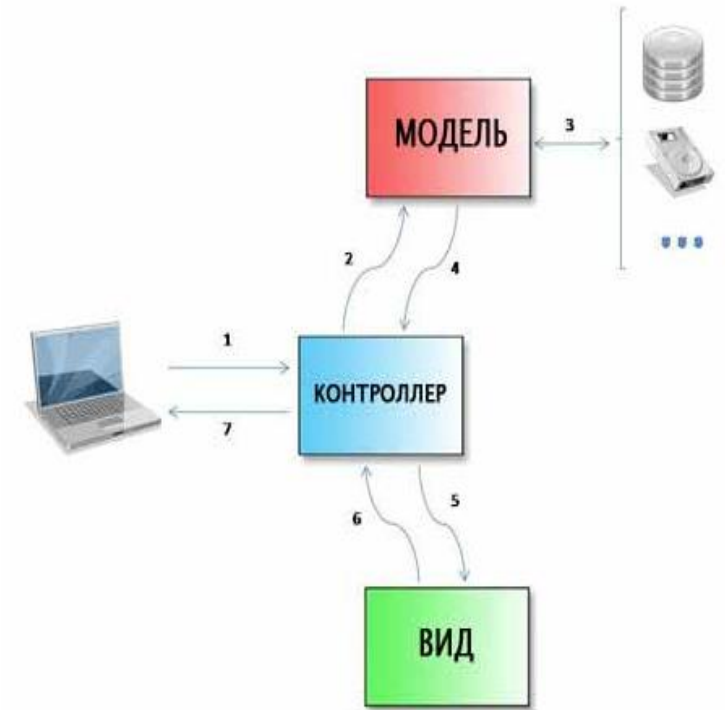
Отображение данных, оформление и другие аспекты презентации модели

Контроллер (Controller)

Реагирует на действия пользователя, интерпретирует данные, введенные пользователем, и информирует модель и производит необходимые манипуляции с моделью и видом.

MVC

- Контроллер получает запрос пользователя [1].
- Контроллер проверяет запрос и параметры, а затем вызывает модель, запрашивая у неё список доступных книг по теме *фэнтези* [2].
- Модель получает данные из базы (или из другого источника, в котором хранится информация) [3], применяет фильтры и необходимую логику, а затем возвращает данные, которые представляют список книг [4].
- Контроллер использует подходящий вид [5] для представления данных пользователю [6-7]. Если запрос приходит с мобильного телефона, используется вид для мобильного телефона; если пользователь использует определённое оформление интерфейса, то выбирается соответствующий вид, и так далее.



API

API (программный интерфейс приложения, интерфейс прикладного программирования) (англ. application programming interface, API [эй-пи-ай]) — описание способов (набор классов, процедур, функций, структур или констант), которыми одна компьютерная программа может взаимодействовать с другой программой.

- Обычно входит в описание какого-либо интернет-протокола (например, RFC), программного каркаса (фреймворка) или стандарта вызовов функций операционной системы.
- Часто реализуется отдельной программной библиотекой или сервисом операционной системы.
- Используется программистами при написании всевозможных приложений.

API как средство интеграции приложений

- API определяет функциональность, которую предоставляет программа (модуль, библиотека), при этом API позволяет абстрагироваться от того, как именно эта функциональность реализована.
- Если программу (модуль, библиотеку) рассматривать как чёрный ящик, то API — это множество «ручек», которые доступны пользователю данного ящика и которые он может вертеть и дёргать.

API в веб-приложениях на примерах

- Некое приложение – например, Github – имеет свой API, которым могут воспользоваться другие разработчики. То, как они будут пользоваться им зависит от возможностей, которые предоставляет API и от того, насколько хорошо работает фантазия у разработчиков. API Гитхаба позволяет, например, получать информацию о пользователе, его аватаре, читателях, репозиториях и множество других полезных и интересных сведений.
- Если взять, к примеру, API Твиттера, то интерфейс этого сервиса может выдать вам информацию о твитах пользователя, его читателей и о тех, кто его читает, и так далее. Это лишь малая часть возможностей, которые любой желающий может воплотить, используя API стороннего сервиса или создавая свой собственный.

API в веб-приложениях на примерах

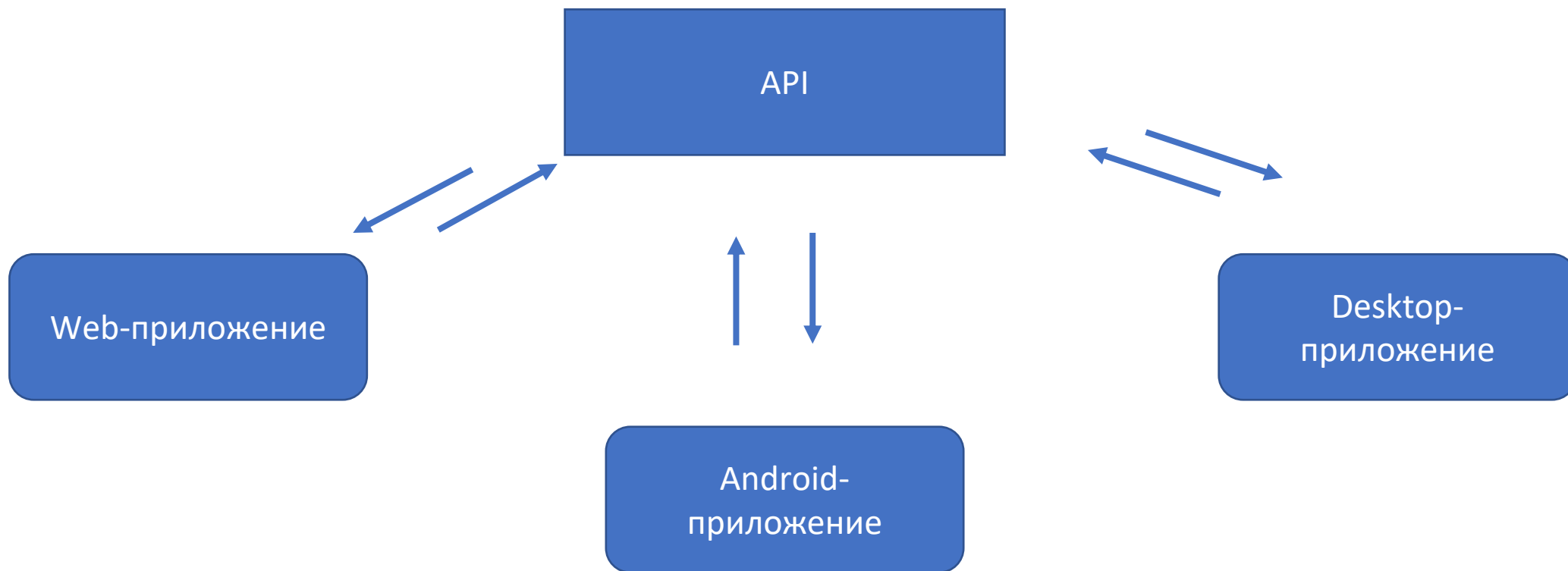
- Стандартный запрос данных от стороннего API выглядит примерно так:
 - curl
<https://api.github.com/users/Freika>
- Ответом на запрос будет примерно такая информация:

```
{  
  "login": "Freika",  
  "id": 3738638,  
  "avatar_url":  
    "https://avatars.githubusercontent.com/u/3738638?v=3",  
  "gravatar_id": "",  
  "site_admin": false,  
  "name": "Evgeniy",  
  "company": "",  
  "blog": "http://frey.su/",  
  "location": "Barnaul",  
  "email": "",  
  "hireable": true,  
  "bio": null,  
  "public_repos": 39,  
  "public_gists": 13,  
  "followers": 15,  
  "following": 21,  
  "created_at": "2013-03-01T13:48:52Z",  
  "updated_at": "2014-12-15T13:55:03Z"  
}
```

Зачем нужен API вашему приложению?

- Существует несколько ситуаций, в которых вы можете захотеть создать API для вашего собственного любовно написанного и отрефакторенного приложения.
 - Мобильное приложение! Да-да, множество мобильных приложений для различных сервисов работают при использовании API этих самых сервисов. Вы описали API, сделали простенькое мобильное приложение и клиент со смартфоном будет получать информацию в свое устройство именно через API. Это удобно, это разумно, это имеет смысл.
 - Опенсорс. Все становится лучше, если использовать опенсорс :) На самом деле, если у вашего приложения сложилась определенная аудитория, которая пользуется им, почему бы не обернуть это себе на пользу? Ну и на пользу аудитории, конечно же, тоже. Создайте API, при помощи которого ваши пользователи при желании смогут создать новые клиенты для вашего приложения, новые сервисы на его основе и, быть может, раскроют новые его грани.
 - Максимальное разделение фронтенда и бэкенда. Например, при использовании фронтенд-фреймворков. О том, как подключить фронтенд-приложение на Angular.js к API мы даже написали целую статью.

API



Django (Джанго) — свободный фреймворк для веб-приложений на языке Python, использующий шаблон проектирования MVC

Django Software Foundation (сокращённо DSF) — некоммерческая организация, основана 17 июня 2008 года для поддержки продвижения, разработки, обучения и развития свободного программного каркаса для веб-приложений на языке Python — Django.
https://ru.wikipedia.org/wiki/Django_Software_Foundation

Django (Джанго) <https://ru.wikipedia.org/wiki/Django>

Сайт на Django строится из одного или нескольких приложений, которые рекомендуется делать отчуждаемыми и подключаемыми. Это одно из существенных архитектурных отличий этого фреймворка от некоторых других (например, Ruby on Rails).

Также, в отличие от других фреймворков, обработчики URL в Django конфигурируются явно при помощи регулярных выражений. Для работы с базой данных Django использует собственный ORM, в котором модель данных описывается классами Python, и по ней генерируется схема базы данных.

Модель ролей объекта — методика концептуального проектирования информационных систем, включающая собственную графическую нотацию.

ORM — технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования, создавая «виртуальную объектную базу данных». Существуют как проприетарные, так и свободные реализации этой технологии.

Пример веб-приложения, которое показывает «Hello World!»:

```
1 from django.http import HttpResponse
2
3 def hello(request):
4     return HttpResponse("Hello world")
```

Веб-фреймворк Django используется в сайтах **Instagram, Disqus, Mozilla, The Washington Times, Pinterest, YouTube, Google** и др.

Разработчики

Расселл Кейт-Маги (Russell Keith-Magee)
Адриан Головатый (Adrian Holovaty),
Саймон Виллисон (Simon Willison),
Джейкоб Каплан-Мосс (Jacob Kaplan-Moss),
Уилсон Майнер (Wilson Miner)

Разработчики заявляют, что не обязуются строго придерживаться какой бы то ни было методологии в развитии проекта, предпочитая делать то, что кажется им наиболее логичным

Архитектура

Архитектура Django похожа на «[Модель-Представление-Контроллер](#)» (MVC).

Контроллер классической модели MVC примерно соответствует уровню, который в Django называется Представление (*View*), а презентационная логика Представления реализуется в Django уровнем Шаблонов (*Template*). Из-за этого уровневую архитектуру Django часто называют «Модель-Шаблон-Представление» (MTV).

Архитектура Django

Первоначальная разработка Django как средства для работы новостных ресурсов достаточно сильно отразилась на его архитектуре: он предоставляет ряд средств, которые помогают в быстрой разработке веб-сайтов информационного характера.

Так, например, разработчику не требуется создавать контроллеры и страницы для административной части сайта, в Django есть встроенное приложение для управления содержимым, которое можно включить в любой сайт, сделанный на Django, и которое может управлять сразу несколькими сайтами на одном сервере.

Административное приложение позволяет создавать, изменять и удалять любые объекты наполнения сайта, протоколируя все совершённые действия, и предоставляет интерфейс для управления пользователями и группами (с пообъектным назначением прав).

В дистрибутив Django также включены приложения для системы комментариев, синдикации [RSS](#) и [Atom](#), «статических страниц» (которыми можно управлять без необходимости писать контроллеры и представления), перенаправления URL и другое.

Некоторые возможности Django:

1. ORM, API доступа к БД с поддержкой транзакций
2. встроенный интерфейс администратора, с уже имеющимися переводами на многие языки
3. диспетчер URL на основе регулярных выражений
4. расширяемая система шаблонов с тегами и наследованием
5. система кеширования
6. интернационализация
7. подключаемая архитектура приложений, которые можно устанавливать на любые Django-сайты
8. «generic views» — шаблоны функций контроллеров
9. авторизация и аутентификация, подключение внешних модулей аутентификации: LDAP, OpenID и проч.
10. система фильтров («middleware») для построения дополнительных обработчиков запросов, как например включённые в дистрибутив фильтры для кеширования, сжатия, нормализации URL и поддержки анонимных сессий
11. библиотека для работы с формами (наследование, построение форм по существующей модели БД)
12. встроенная автоматическая документация по тегам шаблонов и моделям данных, доступная через административное приложение

На базе Django разработан ряд готовых решений со свободной лицензией, среди которых интернет-магазины, системы управления содержимым, а также более узконаправленные проекты.

Конфигурация сервера

Django проектировался для работы под управлением [Apache](#) с модулем [mod_python](#) и с использованием [PostgreSQL](#) в качестве базы данных.

С включением поддержки [WSGI](#), Django может работать под управлением [FastCGI](#), [mod_wsgi](#), или [SCGI](#) на Apache и других серверах ([lighttpd](#), [nginx](#),...), сервера [uWSGI](#).

В настоящее время, помимо базы данных [PostgreSQL](#), Django может работать с другими [СУБД](#): [MySQL](#), [SQLite](#), [Microsoft SQL Server](#), [DB2](#), [Firebird](#), SQL Anywhere и [Oracle](#).

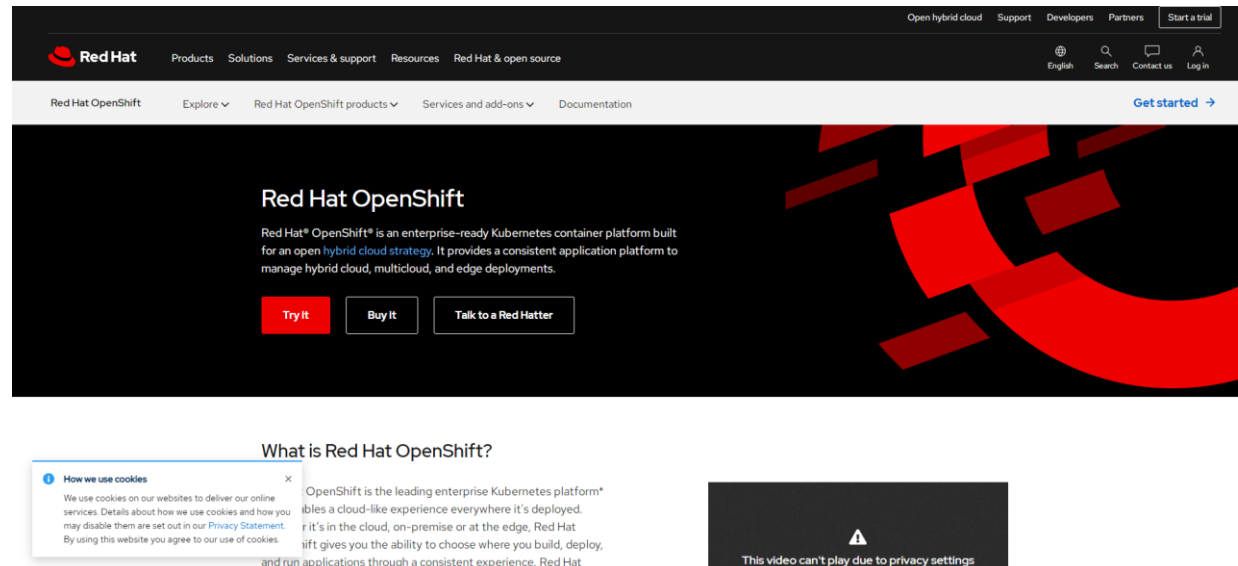
В составе Django присутствует собственный [веб-сервер](#) для разработки. Сервер автоматически определяет изменения в файлах исходного кода проекта и перезапускается, что ускоряет процесс разработки на [Python](#). Но при этом он работает в [однопоточном](#) режиме и пригоден только для процесса разработки и отладки приложения.

Хостинг для DJANGO

Django можно развернуть на [PaaS](#)-сервисах [RedHat](#):

- [OpenShift](#), в том числе и бесплатно
- [Heroku](#)
- На хостинге [PythonAnywhere](#)

<https://www.redhat.com/en/technologies/cloud-computing/openshift>



Хостинг для DJANGO

Django на хостинге Google

[Google App Engine](#) Python позволяет использовать для работы любую версию Django. Существуют некоторые особенности при работе с Django в Google App Engine Python 2.7.

- Django ORM несовместим с хранилищем App Engine Datastore, который представляет собой NoSQL хранилище. Это в свою очередь делает ограниченной работу определённых функций фреймворка, в частности, встроенного административного интерфейса.
- Также с Django можно работать через Google cloud-sql. Это позволяет использовать все возможности фреймворка, как и при работе с обычными реляционными базами.

Существует community-версия Django-nonrel, адаптированная для работы с нереляционными хранилищами данных, которая позволяет полноценно (насколько это возможно технически) использовать Django на платформе [Google App Engine](#). А подключаемое приложение django-dbindexer позволяет даже эмулировать JOIN в нереляционных хранилищах данных.

Google App Engine

Материал из Википедии — свободной энциклопедии

[править | править код]

Текущая версия страницы пока **не проверялась** опытными участниками и может значительно отличаться от **версии**, проверенной 19 июля 2019; проверки требуют **4 правки**.

Google App Engine — служба *хостинга* сайтов и *web-приложений* на серверах Google с бесплатным именем `<имя_сайта>.appspot.com` либо с *собственным именем*, задействованным с помощью *служб Google*.

App Engine представлена в апреле 2008, доступны как бесплатные учётные записи:^[1] «…до 1 Гб дискового пространства, 10 Гб входящего трафика в день, 10 Гб исходящего трафика в день, 200 миллионов гигациклов CPU в день и 2000 операций отправления электронной почты в день»,^[2] так и возможность приобретения дополнительных ресурсов^[3].

Приложения, разворачиваемые на базе App Engine, должны быть написаны на Python, Java, Node.JS, .NET, Ruby, Go либо PHP. Последние два до сих пор имеют статус «экспериментальных» инструментов, хотя Google и заявлял о намерениях расширить список поддерживаемых языков программирования и сред, а саму службу сделать не зависящей от какого-либо языка программирования. Среда исполнения Python^[4] включает в себя полную реализацию возможностей самого Python, большинство функций *стандартной библиотеки языка*, ограниченную версию Django, и т. д.

Предлагается набор API для служб хранилища, datastore API (BigTable),^[5] аккаунтов Google,^[6] загрузки данных по URL,^[7] электронной почты^[8] и т. д.

Предоставлена возможность использовать *планировщик задач cron* как для приложений, реализованных на Python, так и на Java. Разрешено планирование не более 20 заданий.^{[9][10]}

Использование службы учётных записей Google позволяет быстро начать работу с приложением, нет необходимости проводить отдельную регистрацию учётных данных на каждом сайте. Это также позволяет разработчику не заботиться о реализации ещё одной системы регистрации пользователей специально для своего приложения.

Платформа Google конкурирует с *аналогичными сервисами* от Amazon, которые предоставляют возможности размещать файлы и веб-приложения, используя свою инфраструктуру.

Содержание [скрыть]
1 Отличия от традиционной модели хостинга приложений
2 Ограничения
3 Пример самого маленького приложения

Google App Engine



Google App Engine

Application: your-version-1.0

Charts

Resources

Application Settings

Application Status

Application Logs

Application Settings

Application Status

Application Logs

Тип	среда разработки
Разработчик	Google
Написана на	Python
Операционная система	GNU/Linux
Последняя версия	1.9.6 (3 июня 2014)
Лицензия	GPLv2
Сайт	appengine.google.com

Медиафайлы на Викискладе

www.djangopackages.com
<https://djangopackages.org/>

Помимо возможностей, встроенных в ядро фреймворка, существуют пакеты, расширяющие его возможности. Возможности, предоставляемые пакетами, а также полный перечень пакетов удобно отслеживать через специальный ресурс — www.djangopackages.com (<https://djangopackages.org/>)

The screenshot shows the Django Packages website. At the top is a dark green header with the Django Packages logo, a search bar, and a 'Log In' link. Below the header is a grid of category links including ACTIVITIES, ADMIN INTERFACE, ANALYTICS, ANTI SPAM, API CREATION, ASSET MANAGERS, AUTHENTICATION, AUTHORIZATION, AUTO-COMplete, AUTO-UPGRADE, AWARDS AND BADGES, BLOGS, CACHING, CALENDAR, CAPTCHA, CHAT, CMS, COMMENTING, CONFIGURATION, COOKIECUTTERS, COUNTRIES, CUSTOM MODELS, DATA TOOLS, DEPLOYMENT, DESIGN, DEVELOPER TOOLS, DJANGO-CMS, DJANGO REST FRAM..., DJANGO SHOP PLUG..., DOCUMENT MANAGEM..., E-COM, E-COMMERCE, EMAIL, ERROR HANDLING, and FEED AGGREGATORS. Below this is a light gray section with the text 'Django Packages is a directory of reusable apps, sites, tools, and more for your Django projects.' and two buttons: 'Add Package' and 'Add Grid'. The main content area is divided into four columns: 'Apps' (Small components used to build projects. An app is anything that is installed by placing in settings.INSTALLED_APPS. Show Apps (3787)), 'Projects' (This is for individual projects such as Django Packages, DjangoProject.com, and others. Show Projects (196)), 'Frameworks' (Large efforts that combine many python modules or apps. Examples include Django, django-cms, and Mezzanine. Most CMses fall into this category, and so do storefronts. Show Frameworks (168)), and 'Other' (Other are not installed by settings.INSTALLED_APPS, are not frameworks or sites but still help Django in some way. Show Other (665)). At the bottom, there are three sections: 'Latest Python 3 Package Releases', 'Latest 5 Packages Added', and 'Random 5'.

https://djangopackages.org/

The screenshot shows the Django Packages website interface. At the top, the header includes the Django Packages logo and a search bar. A navigation menu lists various categories such as ACTIVITIES, AUTHORIZATION, CAPTCHA, and more. The main content area features a large heading: "Django Packages is a directory of reusable apps, sites, tools, and more." Below this heading are two prominent green buttons: "Add Package" and "Add Grid", which are highlighted with a red box. A red arrow points from this box to an OAuth authorization overlay on the right side of the screen. The overlay, titled "Authorize Django Packages", shows a connection between the Django Packages logo and a GitHub user profile. It states that "Django Packages by pydanny" wants to access the user's account with "Public data only" (limited access to public data). The overlay includes "Cancel" and "Authorize pydanny" buttons, and a note that authorization will redirect to https://djangopackages.org/. At the bottom of the overlay, it indicates the app is "Not owned or operated by GitHub", was "Created 11 years ago", and is used by "More than 1K GitHub users".

Django Packages

Search

ACTIVITIES
ADMIN INTERFACE
ANALYTICS
ANTI SPAM
API CREATION
ASSET MANAGERS
AUTHENTICATION

AUTHORIZATION
AUTO-COMPLETE
AUTO-UPGRADE
AWARDS AND BADGES
BLOGS
CACHING
CALENDAR

CAPTCHA
CHAT
CMS
COMMENTING
CONFIGURATION
COOKIECUTTERS
COUNTRIES

CUSTOMIZABLE
DATA
DEFINITION
DEVELOPMENT
DJANGO
DJANGO

Django Packages is a directory of reusable apps, sites, tools, and more.

Add Package **Add Grid**

Apps
Small components used to build projects. An app is anything that is installed by placing in settings.INSTALLED_APPS.
[Show Apps \(3787\)](#)

Projects
This is for individual projects such as Django Packages, DjangoProject.com, and others.
[Show Projects \(196\)](#)

Frameworks
Large efforts that combine many small reusable components. Examples include Django, django CMSes fall into this category.
[Show Frameworks \(196\)](#)

Authorize Django Packages

Django Packages by pydanny wants to access your aiv123aiv account

Public data only
Limited access to your public data

Cancel **Authorize pydanny**

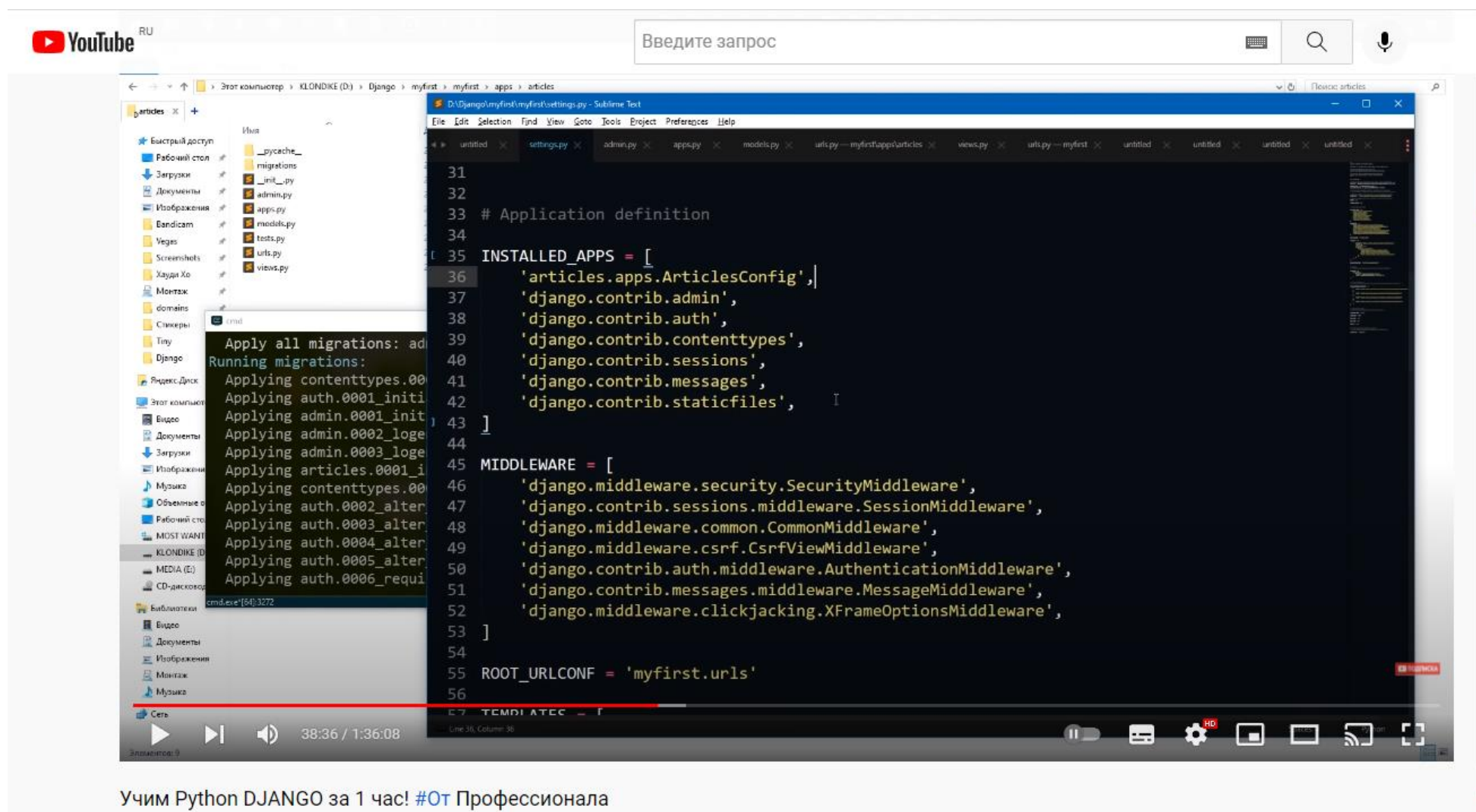
Authorizing will redirect to https://djangopackages.org

Not owned or operated by GitHub | Created 11 years ago | More than 1K GitHub users

[Learn more about OAuth](#)

Учим Python DJANGO за 1 час

<https://www.youtube.com/watch?v=w4nrT7emiVc>



The screenshot displays a YouTube video interface. The video player shows a file explorer on the left with a terminal window open, displaying migration commands. The main window shows a code editor with Django settings.py, including application definitions and middleware configurations.

Terminal output:

```
Apply all migrations: admin
Running migrations:
  Applying contenttypes.0001_initi...
  Applying auth.0001_initi...
  Applying admin.0001_initi...
  Applying admin.0002_loge...
  Applying admin.0003_loge...
  Applying articles.0001_i...
  Applying contenttypes.000...
  Applying auth.0002_alter...
  Applying auth.0003_alter...
  Applying auth.0004_alter...
  Applying auth.0005_alter...
  Applying auth.0006_requi...
```

Code editor content (settings.py):

```
31
32
33 # Application definition
34
35 INSTALLED_APPS = [
36     'articles.apps.ArticlesConfig',
37     'django.contrib.admin',
38     'django.contrib.auth',
39     'django.contrib.contenttypes',
40     'django.contrib.sessions',
41     'django.contrib.messages',
42     'django.contrib.staticfiles',
43 ]
44
45 MIDDLEWARE = [
46     'django.middleware.security.SecurityMiddleware',
47     'django.contrib.sessions.middleware.SessionMiddleware',
48     'django.middleware.common.CommonMiddleware',
49     'django.middleware.csrf.CsrfViewMiddleware',
50     'django.contrib.auth.middleware.AuthenticationMiddleware',
51     'django.contrib.messages.middleware.MessageMiddleware',
52     'django.middleware.clickjacking.XFrameOptionsMiddleware',
53 ]
54
55 ROOT_URLCONF = 'myfirst.urls'
56
```

Video player controls show the video is at 38:36 / 1:36:08.

Учим Python DJANGO за 1 час! #От Профессионала

Учим Python DJANGO

Продолжение следует...

Лекция 6

Темы

2) Работа с сокетами

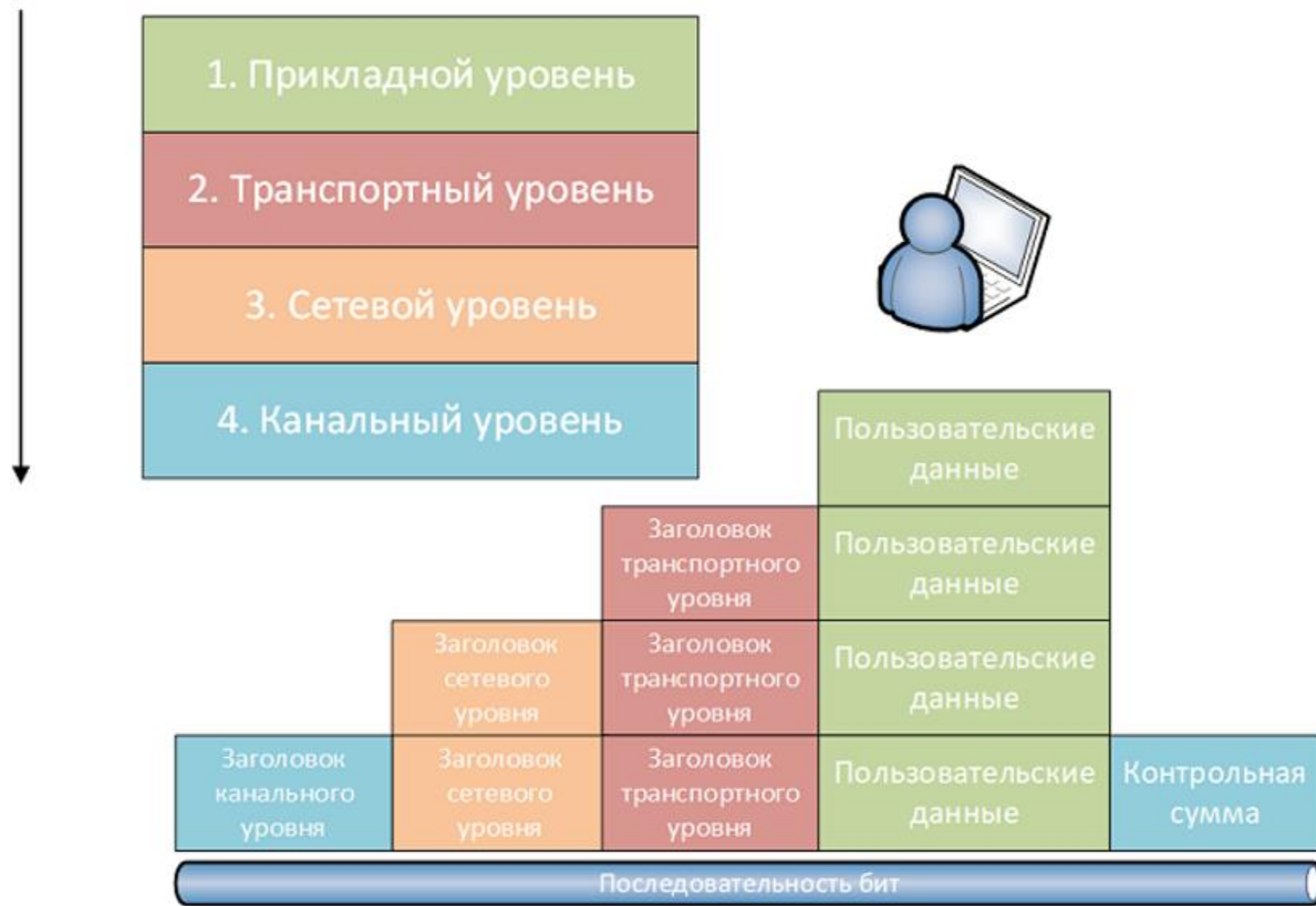


Рисунок 1 – Сетевая модель

Сóкет (socket — разъём) — название программного интерфейса для обеспечения обмена данными между процессами. Процессы при таком обмене могут исполняться как на одной ЭВМ, так и на различных ЭВМ, связанных между собой сетью. **Сокет** — абстрактный объект, представляющий конечную точку соединения.

Следует различать клиентские и серверные сокеты.

Клиентские сокеты можно сравнить с конечными аппаратами телефонной сети, а серверные — с коммутаторами.

Клиентское приложение (например, браузер) использует только клиентские сокеты, а серверное (например, веб-сервер, которому браузер посылает запросы) — как клиентские, так и серверные сокеты.

Интерфейс сокетов впервые появился в BSD Unix. Программный интерфейс сокетов описан в стандарте POSIX.1 и в той или иной мере поддерживается всеми современными операционными системами.

Принципы сокетов

Для взаимодействия между машинами с помощью стека протоколов [TCP/IP](#) используются адреса и порты. Адрес представляет собой 32-битную структуру для протокола [IPv4](#), 128-битную для [IPv6](#). Номер порта — целое число в диапазоне от 0 до 65535 (для протокола [TCP](#)). Эта пара определяет сокет («гнездо», соответствующее [адресу](#) и [порту](#)).

В процессе обмена, как правило, используется два сокета — сокет отправителя и сокет получателя. Например, при обращении к серверу на [HTTP](#)-порт сокет будет выглядеть так: 194.106.118.30:80, а ответ будет поступать на mmm.nnn.prrr.qqq: xxxx.

Каждый [процесс](#) может создать «слушающий» сокет (серверный сокет) и *привязать* его к какому-нибудь [порту](#) операционной системы (в [UNIX](#) непривилегированные процессы не могут использовать порты меньше 1024).

Принципы сокетов

Слушающий процесс обычно находится в цикле ожидания, то есть просыпается при появлении нового соединения. При этом сохраняется возможность проверить наличие соединений на данный момент, установить тайм-аут для операции и т. д.

Каждый сокет имеет свой адрес. ОС семейства UNIX могут поддерживать много типов адресов, но обязательными являются [INET-адрес](#) и [UNIX-адрес](#). Если привязать сокет к UNIX-адресу, то будет создан специальный файл (*файл сокета*) по заданному пути, через который смогут общаться любые локальные процессы путём чтения/записи из него (см. [сокет домена Unix](#)). Сокеты типа [INET](#) доступны из сети и требуют выделения номера порта.

Обычно клиент явно «подсоединяется» к слушателю, после чего любое чтение или запись через его [файловый дескриптор](#) будут передавать данные между ним и сервером.

Итоги лекции

Вопросы ?